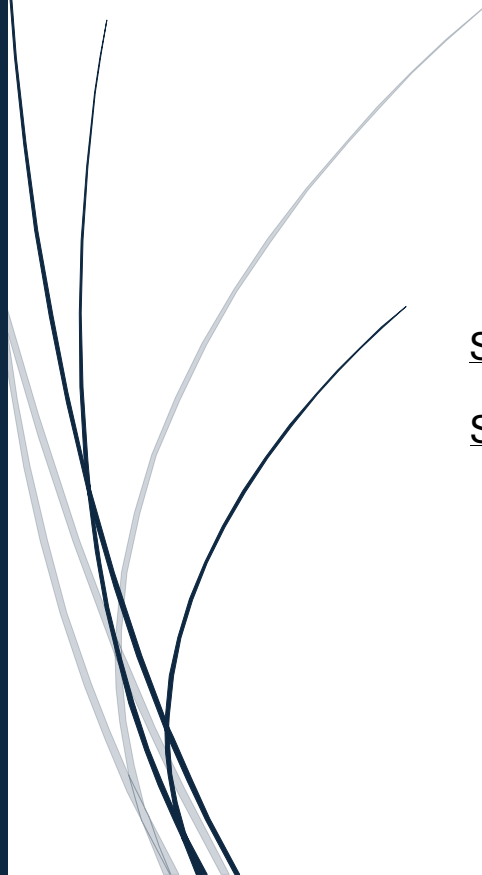


9/15/2025

High Performance Computing with GPUs

HPC Assignment 1



Submitted by: Moiz Ansari [23i-0523]

Submitted to: Dr. Imran Ashraf

Table of Contents

1	Introduction	2
2	Implementation Details	2
2.1	CPU Implementation	2
2.2	GPU Implementation	2
2.2.1	Memory Allocation and Transfer	2
2.2.2	1D Kernel Implementation	2
2.2.3	2D Kernel Implementation	3
2.3	Result Verification	3
3	Performance Analysis	3
3.1	Measurement Setup	3
3.2	Results Table.....	4
3.2.1	1D Kernel.....	4
3.2.2	2D Kernel.....	4
3.3	Graphs	4
3.3.1	Graph 1: Kernel execution time vs. matrix size	4
3.3.2	Graph 2: Total GPU time vs. matrix size.....	5
3.3.3	Graph 3: Speedup vs. matrix size	5
4	Analysis and Observations	5
4.1	CPU vs. GPU	5
4.2	1D vs. 2D Kernel	5
4.3	Different GPUs (Colab vs. Lab GPU)	6
5	Conclusion	6

1 Introduction

The purpose of this assignment is to implement and analyze matrix multiplication on both CPU and GPU using CUDA, comparing execution time, data transfer, and speedup. GPUs exploit massive parallelism, while CPUs handle mostly serial tasks, making them less efficient for large-scale matrix operations.

Experiments were performed on both Lab GPU and Google Colab GPU, for matrix sizes ranging from 256x256 to 2048x2048, measuring CPU time, GPU time (kernel + transfers), and speedup using CUDA events, which are presented through tables and graphs.

2 Implementation Details

2.1 CPU Implementation

The matrix multiplication code for CPU execution is implemented using 3 nested loops in C. The first two loops iterate over the rows of the first matrix and columns of the second matrix, whereas the third loop calculates the dot product of them by iterating over each element of the row-column pair.

This method results in time complexity of $O(n^3)$ for an $n \times n$ matrix. This is a simple approach and works normally for small matrices, but not suitable for large values of N , as it becomes significantly slower as matrix size increases.

2.2 GPU Implementation

The GPU version of matrix multiplication was implemented using CUDA. The implementation involved explicit memory management, kernel launches, and the use of both **1D and 2D kernel configurations**.

2.2.1 Memory Allocation and Transfer

For GPU implementation, first, memory was allocated in the device for all matrices A, B and C using `cudaMalloc`. Then, the input matrices A and B were initialized on host memory and were transferred to device using `cudaMemcpy`. Then after the kernel execution, the result matrix C was transferred back to host memory using `cudaMemcpy`. CUDA events were used to measure the time for host-to-device transfer, device-to-host transfer, kernel execution, and total GPU execution time.

2.2.2 1D Kernel Implementation

In the 1D kernel implementation, a linear thread index was computed as **$index = threadIdx.x + blockIdx.x * blockDim.x$** . The index was then mapped to 2D coordinates: **$row = index / N$** , **$col = index \% N$** . Each thread computed a single element $C[row][col]$ by performing the dot product of the corresponding row of A and column of B.

Grid and Block Configuration:

- Number of threads per block: 512
- Number of blocks per grid: $(N * N) / \text{threadsPerBlock}$

Out of all the sizes, 512 threads per block gave the best results on all sizes of matrices.

2.2.3 2D Kernel Implementation

In the 2D kernel implementation, row and column indices were directly computed from **block and thread coordinates**:

row = threadIdx.x + blockIdx.x * blockDim.x

col = threadIdx.y + blockIdx.y * blockDim.y

Each thread computed a single element of the result matrix $C[\text{row}][\text{col}]$.

Grid and Block Configuration:

- Number of threads per block: threadsPerBlock (32, 32)
- Number of blocks per grid: $(N + \text{threadsPerBlock} - 1) / \text{threadsPerBlock}$

The improvement of performance on 32x32 threads per block was observable on larger matrices so that's why this size was selected.

2.3 Result Verification

After the computation of matrix multiplication on CPU and GPU, the results are then passed inside the verification function. This function checks if the difference of both results is no more than $1e-4$, which is the tolerance of error. In my implementation, all results were matched successfully.

3 Performance Analysis

3.1 Measurement Setup

- **Cuda Events for Time Recording**

To measure execution times accurately on the GPU, CUDA events (cudaEvent_t) were used. CUDA events provide timestamps directly from the GPU, ensuring that measurements are not affected by CPU–GPU synchronization delays.

- **Measured Metrics**

The following timings were recorded for each experiment:

CPU execution time: Time taken by sequential CPU execution of matrix multiplication.

Data transfer device-to-host (D2H): Time taken to copy initial matrices from device to host memory.

GPU kernel execution time: Time taken by CUDA kernel to perform matrix multiplication on GPU.

Data transfer host-to-device (H2D): Time taken to copy results from host to device.

Total GPU time: Total time taken by GPU (D2H + Kernel + H2D).

- **Experimental Conditions**

Both kernel 1D and 2D configurations were tested. The experiment was performed repeatedly to calculate the average of all results and exclude the outliers caused by unconditional factors.

The experiment was performed on both Lab GPU and Google Colab GPU for comparison.

3.2 Results Table

3.2.1 1D Kernel

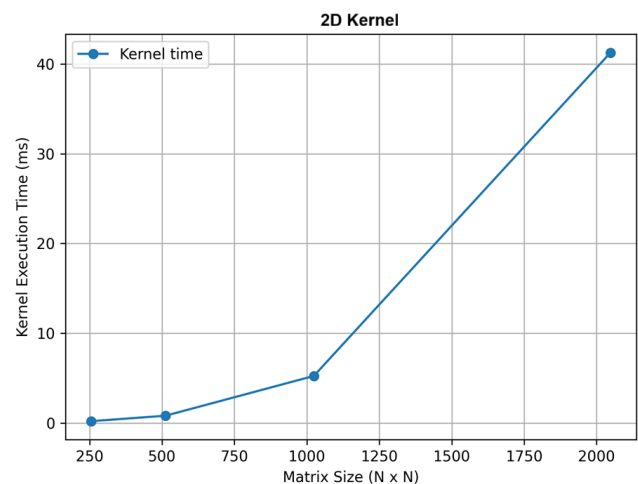
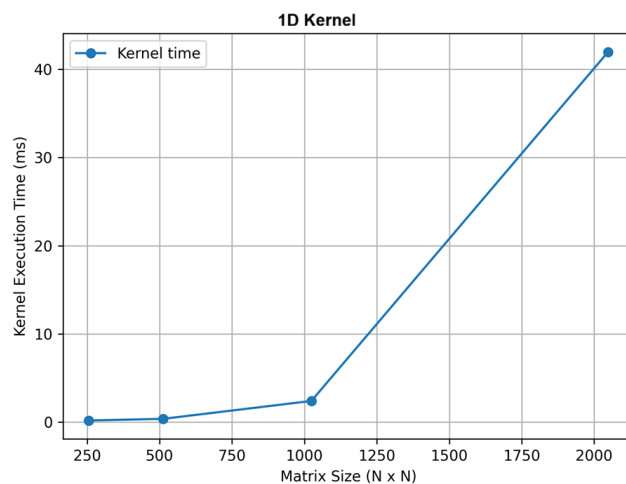
Size	Avg CPU	Avg H2D	Avg Kernel	Avg D2H	Avg Total GPU	Speedup Kernel	Speedup Total
256	80.8135	0.1871	0.2809	0.2269	0.7311	287.6760	110.5440
512	752.3635	0.7139	1.0307	0.8086	2.5906	729.9876	290.4261
1024	8589.8392	1.9870	7.2635	2.9672	12.2387	1182.6079	701.8599
2048	106289.8328	7.8532	54.5959	11.8298	74.1903	1946.8449	1432.6652

3.2.2 2D Kernel

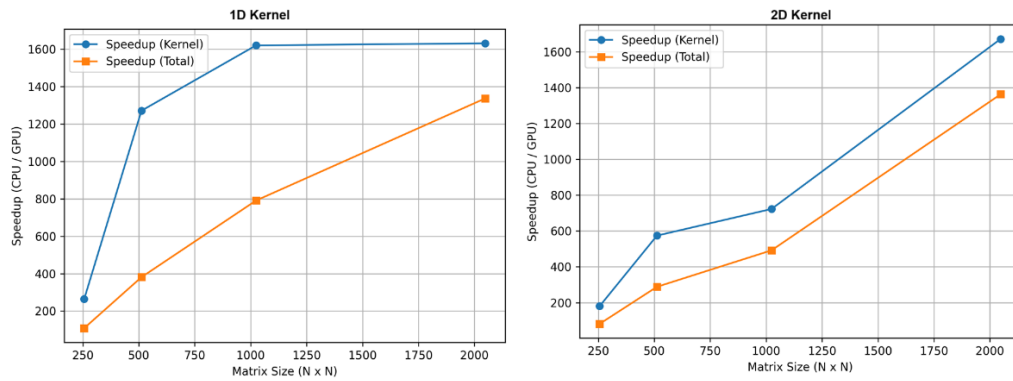
Size	Avg CPU	Avg H2D	Avg Kernel	Avg D2H	Avg Total GPU	Speedup Kernel	Speedup Total
256	54.1199	0.1806	0.2565	0.2208	0.6924	210.9891	78.1656
512	756.5035	0.7037	1.0402	0.8170	2.5988	727.2904	291.0966
1024	8042.4562	1.9809	6.8934	2.9142	11.8434	1166.6934	679.0642
2048	101811.9594	7.6666	53.3128	11.7652	72.8345	1909.7101	1397.8533

3.3 Graphs

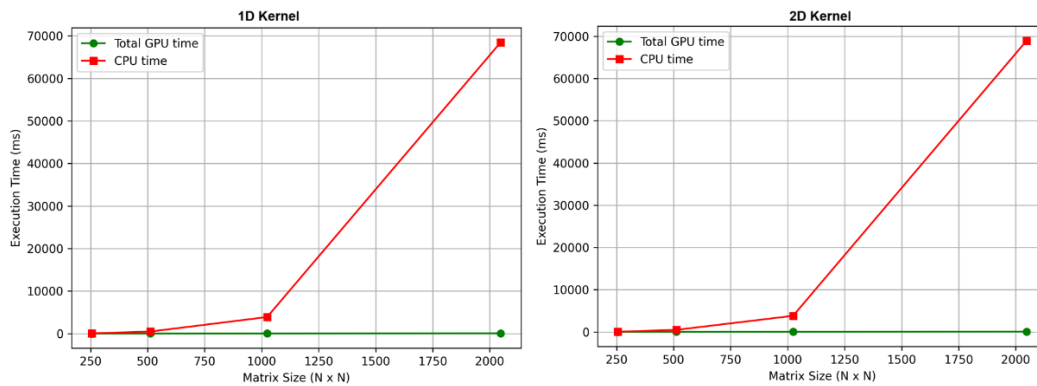
3.3.1 Graph 1: Kernel execution time vs. matrix size



3.3.2 Graph 2: Total GPU time vs. matrix size



3.3.3 Graph 3: Speedup vs. matrix size



4 Analysis and Observations

4.1 CPU vs. GPU

For the starting values of N (like upto 32), the CPU and GPU performance difference is negligible and almost equivalent to each other. But as the value of N starts to increase, a significant amount of change can be observed in the efficient GPU performance compared to CPU performance. This is because for small sizes of matrices, the CPU might be competitive due to excessive memory transfer times for GPU.

4.2 1D vs. 2D Kernel

Size	1D Kernel (ms)	2D Kernel (ms)	Faster
256	0.2809	0.2565	2d
512	1.0307	1.0402	1d
1024	7.2635	6.8934	2d
2048	54.5959	53.3128	2d

From the table we can observe that 2D kernel is slightly faster than 1D kernel, but this difference increases as the size of matrices increases. The reason is that 2D mapping matches matrix layout

naturally, so the threads in same wrap access consecutive memory easily. In addition, the 1D kernel wastes parallelism because it must compute $\text{row} = \text{index} / N$, $\text{col} = \text{index} \% N$, which adds extra division/mod operations.

4.3 Different GPUs (Colab vs. Lab GPU)

Size	Colab T4 Kernel (ms)	Lab GPU Kernel (ms)	Faster
256	0.2687	0.2081	Lab
512	1.03545	0.6033	Lab
1024	7.07845	3.8305	Lab
2048	53.95435	41.6088	Lab

Analysis of Results:

- For smaller matrix sizes, the kernel execution times are already very low on both devices, so the performance gap is not significant.
- For larger matrix sizes, the Lab GPU consistently outperforms the Tesla T4. This advantage comes from its **greater parallelism (more CUDA cores/SMs)** and **higher memory throughput**, which help sustain the heavy computational and memory demands of large-scale matrix multiplication.

Reflection:

- The algorithm of code remains same, yet the **hardware architecture** impacts directly on the performance.
- GPUs with **higher core counts and bandwidth** deliver significant improvements for large workloads, demonstrating that both **algorithm design and hardware capability** must be considered when analyzing performance.

5 Conclusion

The experiments clearly show that **GPU acceleration provides significant speedups over CPU execution**, especially for larger matrices. For small inputs (256, 512), the CPU was competitive due to GPU overheads (data transfers + kernel launch). However, beyond size 1024, the GPU consistently outperformed the CPU by large margins.

Among the two kernels, **2D kernel was generally faster** than 1D kernel due to its natural row/column mapping and more efficient memory access.

Finally, hardware differences also played a role: the **Lab GPU outperformed Colab's Tesla T4** for larger matrices due to higher memory bandwidth and more SMs, showing how GPU architecture directly impacts performance.

Hence, GPU based matrix multiplication is most beneficial for larger workloads, with 2D kernels on high-performance GPUs giving the best results.